

CHaNAS: Coordinated Search for Network Architecture and Scheduling Policy

Weiwei Chen

chenweiwei@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

University of Chinese Academy of
Sciences
Beijing, China

Ying Wang*

wangying2009@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

Gangliang Lin

lingangliang17@mails.ucas.ac.cn

University of Chinese Academy of
Sciences
Beijing, China

Chengsi Gao

gaochengsi17z@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

University of Chinese Academy of
Sciences
Beijing, China

Cheng Liu

liucheng@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

Lei Zhang†

zlei@ict.ac.cn

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

Abstract

Automatically design an efficient DNN solution for a given deep learning task on the target hardware mainly decided by the neural network architecture and the schedule mapping strategy, where the two goals are closely coupled with each other to fully exploit the advantages of the underlying hardware. Prior hardware-aware Neural Architecture Search (NAS) methods mostly ignore the impacts of different scheduling policies (e.g., graph-level optimization, loop transformations, parallelization, etc.) on network candidates being evaluated in the search process. Thus, they may miss the true-optimal architecture that can only be discovered by trying-out different scheduling policies. This work proposes a NAS framework (CHaNAS) that searches for not only the network architecture but also the dedicated scheduling policy, as the optimal co-design solution on target hardware that fully exploits the advantages of the underlying hardware. We propose to use a block-based pre-scheduling methodology to

reduce the co-design search space, and enable the automatic generation of the optimal co-design, including the network architecture and the tensor programs that practice the scheduling policy. We evaluate CHaNAS on Imagenet on different hardware back-ends against the state-of-the-art hardware-aware search method MobileNet-v3. Experimental results show that the co-design solutions obtained by CHaNAS show up to 1.6x, 1.9x, and 1.7x performance boost on NVIDIA P100 GPU, Intel Xeon 8163 CPU, and Samsung Note 10 Mobile, respectively, over the baselines of the same-level accuracy.

CCS Concepts: • Computing methodologies → Machine learning; • Software and its engineering → Software notations and tools.

Keywords: NN-Scheduling Co-design, Hardware-aware Neural Architecture Search, Compiler optimization

ACM Reference Format:

Weiwei Chen, Ying Wang, Gangliang Lin, Chengsi Gao, Cheng Liu, and Lei Zhang. 2021. CHaNAS: Coordinated Search for Network Architecture and Scheduling Policy. In *Proceedings of the 22nd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '21)*, June 22, 2021, Virtual, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3461648.3463846>

1 Introduction

The proliferation of deep learning applications has stimulated the development of efficient neural network architecture design and compilation strategy of Deep Neural Network (DNN) models for different hardware. When implementing an efficient application or domain-specific neural

*Corresponding Author

†Lei Zhang is also with Jeejio (Ningbo) Technology Co., Ltd

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

LCTES '21, June 22, 2021, Virtual, Canada

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8472-8/21/06...\$15.00

<https://doi.org/10.1145/3461648.3463846>

network system [6, 17], there is a deep stack of optimization technologies available, including improved neural network architecture [23–25, 47], optimized frameworks and compilers [4, 12, 18, 35, 40], and even specialized hardware design [16, 41, 45, 46]. However, instead of practising the DNN optimization technologies independently, it has been proved that cross-stack co-design approaches that orchestrate the techniques from different levels can achieve better results [5, 14, 29, 45]. For example, some recent works [14, 21, 22, 27, 33] have investigated how to design an optimized neural network architecture that fully considers the characteristics of the target hardware. Such a hardware-aware network architecture design method, i.e., hardware-aware NAS, can automate the DNN design and even outperform previous human-designed models by involving hardware features into the NAS loop.

Prior hardware-aware NAS only consider the co-optimization of DNN architecture and hardware-related design options such as the representation precision and provision of hardware resources [29, 38, 44]. However, as we have emphasized, the efficiency of DNN applications also depends on multiple layers of optimization techniques. Specifically, in addition to neural network architectures, at the compiler-level, how to schedule the neural network onto the hardware, e.g., task graph reordering, loop reordering, loop tiling, memory customization, and the other compiler-managed scheduling policies, also plays an essential role in the performance of the target neural network system [15, 19, 28]. For example, a typical hardware-aware NAS framework usually relies on a performance model to estimate the performance of the candidate neural networks on the target hardware, and then decides whether the discovered network should be kept or updated during the iterative search process. However, in these evaluation-and-search iterations, almost all prior hardware-aware NAS works assume a fixed network scheduling policy that may not extract the best performance from the under-evaluation network architecture on the hardware. Some other works focus on tuning the schedule mapping strategy when given a NN model to optimize the performance for different hardware [15, 20, 43], but they ignore the correlation between the interactive stacks and lead to locally-optimal solutions. In contrast, we prove that an ideal hardware-aware NAS framework should consider the compiler-level optimization, i.e., the scheduling policies, for the target hardware, in order to find the optimal solution at the system level. To achieve this purpose, we for the first time propose the Compiler and Hardware aware Network Architecture Search (CHaNAS) framework in this paper.

In CHaNAS, we orchestrate two key components from the deep learning optimization stack: the DNN architecture and the scheduling policy in the compiler that tactically maps a model onto the target hardware. As shown in the Figure 1, compared to traditional methods that either optimize the

neural network using NAS based on a fixed schedule policy from a given deep learning library (e.g., Tensorflow), or tunes the scheduling policy to maximize the inference speed for a given network, *CHaNAS constructs a joint search space that combines the DNN architectures and scheduling-level optimization*. For each candidate network evaluated in the search space, CHaNAS can measure and discover its realistic peak performance that can be revealed by trying-out different schedule policies such as instruction-level scheduling, memory allocation, loop transformations, etc, and then choose the true-optimal one. Therefore, with a larger degree of design freedom than previous schedule-agnostic hardware-aware NAS, it more likely to find the optimal specialized DNN solution for varied tasks and different hardware back-ends. As an automated schedule-aware neural architecture search method, CHaNAS can not only produce the DNN implementation but also generate the corresponding high-performance tensor programs for the potential hardware back-ends.

In summary, this paper makes the following contributions:

- We propose the first automatic joint search methodology for neural network architecture and scheduling policy on different hardware, CHaNAS. By using a block-based hierarchical design representation, CHaNAS constructs a large joint co-design space and is able to locate the optimal solution for the target hardware with specified design constraints.
- We propose an elastic supernet that is employed by CHaNAS to generate child neural network models in the solution search process. The basic building blocks for this supernet can be automatically pre-scheduled and evaluated on the target hardware to expose the optimal performance to the CHaNAS during the search process with little cost. It is more likely to find the true optimal co-design solution instead of the scheduling-oblivious network architectures in prior hardware-aware NAS.
- We divide the co-design search space according to the constraints of deployment scenario to reduce the search cost, and employ an evolutionary search strategy to explore the reduced co-design search space, which can gradually generate higher-quality co-design results for the target hardware platform.
- We evaluate CHaNAS on Imagenet on different hardware back-ends against the state-of-the-art (SOTA) hardware-aware search method MobileNet-v3 [23]. Experiments show that the co-design solutions obtained by CHaNAS show up to 1.6x, 1.9x, and 1.7x performance boost on NVIDIA P100 GPU, Intel Xeon 8163 CPU, and Samsung Note 10 Mobile, respectively, over the baselines of the same-level accuracy.

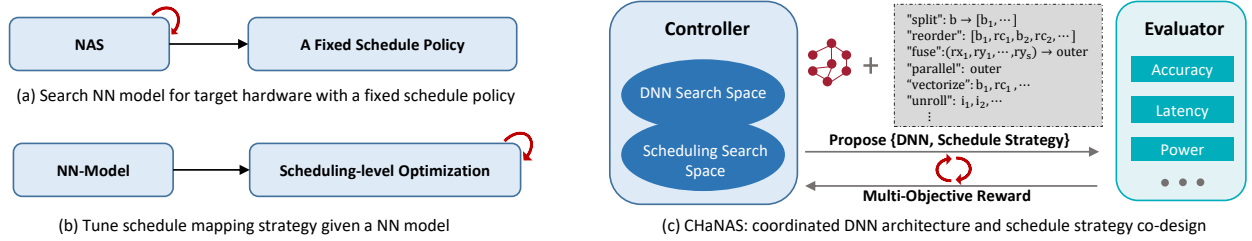


Figure 1. CHaNAS jointly design the neural architecture and the computational-graph scheduler to fit the target hardware characteristics. With an enlarged optimization design search space, CHaNAS is more likely to achieve better results than existing scheduler-agnostic hardware-aware NAS methods.

2 Background & Motivation

In this section, we first introduce two critical factors that impact the DNN solution on the target hardware: the efficient DNN architecture design and the scheduling strategy that maps the DNN on different hardware. Then we present the observation that motivated us to automatically co-design the DNN architecture and corresponding scheduling for the target hardware platforms.

Efficient Neural Network Design. Efficient DNN model design is crucial for the overall performance of the deep learning system. Many efficient neural network architectures are proposed to improve hardware efficiency, such as Squeezenet [25], MobileNets [24], ShuffleNets [47], etc., which mainly focus on reducing the computation (e.g., adopting depth-wise). To reduce the manual effort in the efficient DNN design, Neural architecture search (NAS) tends to automate the architecture design process and begins to dominate the recent efficient network design, while the early NAS methods [30, 50] search for high accuracy architectures without considering hardware efficiency. As the development of NAS techniques [8], researchers begin to combine multi-objective optimization into the NAS process (e.g., latency, energy) [22, 27, 33, 38, 42, 45]. Therefore, those methods take the hardware characteristic into the NAS loop to improve the inference efficiency, but their performance highly depends on the quality of the search space and search strategy [31, 36]. Generally speaking, people follow manual design heuristic for NAS search space design and use either heuristic or ML method for design search.

DNN Model deployment on Hardware. How to map DNNs onto particular hardware devices is crucial to the system performance [7, 32]. Most DNN compilers [12, 18, 37] rely on an intermediate representation (IR) of task graph to abstract the network and then perform operator-level optimization on it. They can either manipulate the network IR at the graph-level, or redefine the implementation of each single operator which is a node in the IR graph. Graph level scheduling techniques include layout optimizations, operator fusion, constant folding, auto-batching, etc. In contrast, operator-level computation optimization is often hardware-specific

and the current are based on some human-written schedule templates, including different schedule primitives (e.g., split, reorder, fuse, etc.) and proper parameters to achieve parallelism, vectorization, memory tilling for different hardware.

Observation: Figure 2 reveals a phenomenon we discovered in the SOTA NAS frameworks that run realistic workloads. For different neural network task specifications, e.g., different data-set, optimization goals, or different performance constraints like the accuracy or latency requirements, an ideal NAS framework is supposed to generate the optimal neural network model for the target hardware through the automated search mechanism. However, as the user-specified constraint of accuracy changes, the Pareto frontiers of the NAS schemes that assume different scheduling solutions will intersect with others, which means none of the NAS baselines in Figure 2 can always generate a better solution than the others when the task specification or constraint changes. For example, in Figure 2, if the designer seeks a neural network that must run faster than 35ms, the NAS scheme assumes *Schedule-B* is better, while the one with *Schedule-A* is more accurate when the latency constraint changes to 20ms. In other words, prior schedule-agnostic NAS technologies cannot guarantee the optimal network solution in the search process as they assume a fixed compiling strategy. Thus, enabling effective and automatic co-optimization of neural network architecture and the scheduling policy is necessary for an optimized neural network system. In this work, we for the first investigate the scheduler and hardware aware NAS that not only searches for the network architecture choices but also the best network scheduling solution on the target hardware at the same time.

3 ChaNAS Framework

3.1 Problem Formalization

CHaNAS explores the massive joint design space, which is the confluence of the DNN architecture space and the network scheduling space for target hardware. The DNN architecture space includes the model hyper-parameters such as the layers type, input size, channels, etc. While the scheduling space contains many compiler-level optimization knobs

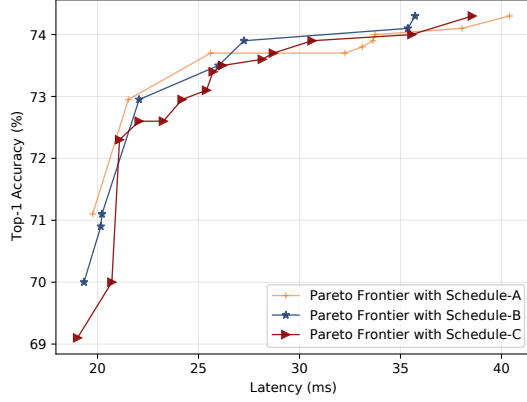


Figure 2. Relative Pareto-frontier of three different scheduling strategies. We randomly extract 200 models from the OFA [9] and test them on the Note 10 mobile, *Schedule-A* presents the original schedule result of TF_Lite, while *Schedule-B* and *Schedule-C* presents two schedule strategies that change the loop split knobs in TF_Lite.

with their parameters, such as loop tiling, operators fusion, reordering and etc. Assuming the DNN architecture search space is $\{arch\} = \{arch_1, \dots, arch_n\}$, and the target design performance constraint is P_{thres} , while the scheduling policy space is $C = \{c_1, \dots, c_m\}$. The objective of CHaNAS is to search from the joint search space $\{\{arch\}, C\}$ for the best DNN architecture $arch^*$ with the associated scheduling strategy $c_{arch^*}^*$, which in together contribute to the maximum network accuracy A and improved performance P that must at least satisfies constraint P_{thres} . In all, We formalize this problem as:

$$\begin{aligned} (arch^*, c_{arch^*}^*) \in & \underset{arch^* \in \{arch\}, c^* \in C}{\operatorname{argmax}} A(arch, c) \\ \text{s.t. } & P(arch^*, c_{arch^*}^*) < P_{thres} \end{aligned} \quad (1)$$

3.2 CHaNAS Overview

As shown in Figure 1(c), CHaNAS has a much larger design space to explore in contrast to prior works that only search for solutions inside the network architecture space $\{arch\}$. Thus, to cover both the DNN model and scheduling policy, it requires a compact design space representation to describe the potential co-design solutions with high efficiency, and also an effective search strategy that quickly converges to high-quality co-design solutions. Thus, in the first place, CHaNAS conducts the architecture/schedule-policy co-design by employing a block-based hierarchical design representation to efficiently reduce the efforts taken to explore unnecessary options.

In the CHaNAS search space, both the network architecture design and the network scheduling is conducted on the basis of neural network blocks, which help divide the entire network design space exploration into two coordinated

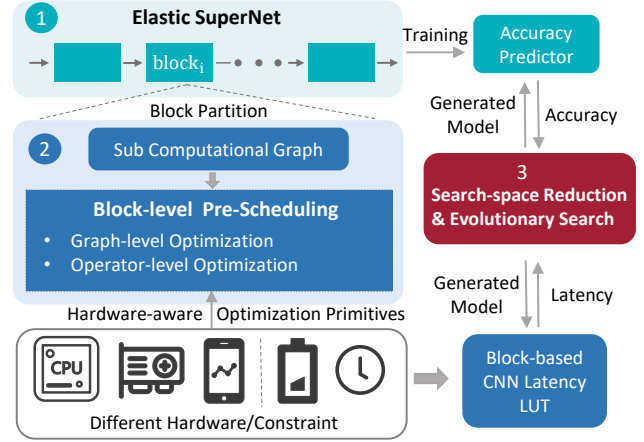


Figure 3. Overview of CHaNAS.

phases. As a sub-graph of interconnected neural network operators, a CHaNAS block is the fundamental unit in both network architecture design and scheduling policy search. For network architecture search, the mainstream NAS approaches will pre-define the elementary blocks and generate the inter-block connection to form a candidate network architecture. At the same time, the scheduler will try to reorder the operators in blocks and then map each block onto hardware by trying-out back-end scheduling techniques such as loop-unrolling and blocking. Therefore, the hierarchical exploration flow of CHaNAS is hinged on neural blocks and involve two coordinated phases: the top-down network scheduling component will pre-generate the implementations for each possible neural block and evaluate them on the target hardware, while the bottom-up network architecture generation unit will search through the possible sequences of the neural blocks that have already been virtually mapped and optimized on hardware by the scheduling unit. With these two exploration paths, the optimal co-design solution can be found at a high probability.

Such a factorized hierarchical search space makes a good balance between the diversity of potential co-design options and the size of the entire co-design search space. Suppose we partition the network into B blocks, and each block has a parameter search space size of R with an average of S scheduling policies per block, then our total search space would be $R * B * S + R^B$, versus the original single-level co-design search space at the size of $R^B * S^B$.

Figure 3 depicts the overview of CHaNAS, which has three major components: (1) An super-net that captures the DNN architecture search space. All candidate DNN architectures can be extracted from the super-net, which achieves fast accuracy evaluation as the candidate DNN directly inherits its parameters to bypass the expensive network training stage; (2) A block-based scheduling space explorer that transforms each block into a computational sub-graph and optimizes it on the target hardware; (3) An search algorithm that divides

the joint space to reduce the search cost and then search within the co-design sub-space ; In CHaNAS, there are three major steps in the co-design flow:

Step 1: Construction of the Elastic Super-Net. We at first build the super-net from which many candidate DNN architectures can be derived. Specifically, each *block* in Super-Net has many variable hyper-parameters such as kernel size, channels, input shape, etc. After the super-net training completes, CHaNAS extracts child networks for the target hardware from the super-net in the search process for co-design solutions.

Step 2: Block-level Pre-Scheduling. For the scheduling-level optimization space, the basic DNN building blocks are virtually pre-scheduled and optimized on the target hardware, so that it generates many block-level co-designs that will be used in the evaluation stage of the global joint design space search. Given a parameterized block, the scheduling space, including the graph-level optimization for overall block topology and the operator-level optimizations that explored via a heuristic method, and the performance of the scheduling policies will be fine-tuned under the direction of a learned cost model until the best scheduling point for that block is obtained.

Step 3: Co-design Exploration. Given the deployment constraints (latency in our test as an example), we first reduce the co-design search space according to the cumulative distribution function of network inference-latency. Then we use an evolutionary-based searcher to explore in the reduced search space, for which we build a DNN accuracy predictor and a performance predictor based on the block performance Look Up Table (LUT) profiled at the block-level pre-scheduling stage. At last, the target DNN model with a dedicated scheduling strategy is returned.

3.3 Elastic Super-net Construction

Block is the basic unit in CHaNAS architecture search and scheduling search. In the CHaNAS design space, if the block structure is defined at an over-fine granularity, i.e., less layers in the block, the graph-level scheduling search space in a block will also be too small to conduct thorough scheduling-level optimization. On the opposite, if the blocks are too large and lead to a massive block design space, it will be less likely for the search algorithm to converge to the optimal architecture in an oversized architecture search space. Therefore, we propose a medium-grained block design method to build an elastic super-net to balance the architecture search efficiency and scheduling search efficacy. The super-net will not induce too much search complexity and contains various large-enough blocks to explore the potential of scheduling-level search.

Figure 4 shows the architecture of super-net, which defines the architecture search space. For each block $block_i$, there are M units included. Following the common practice of NAS approaches [9, 38], we adopt the elastic MBConv cell as

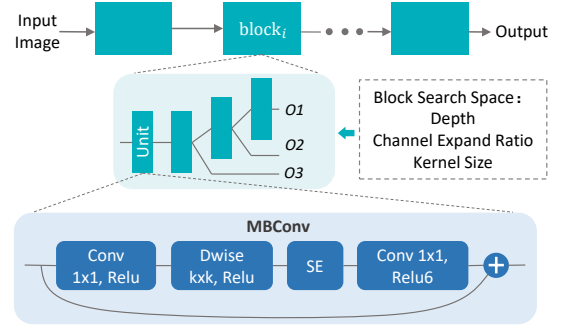


Figure 4. Super-Net architecture, which is composed by a number of blocks.

Table 1. Different scheduling primitives with their parameters

Name	Description & Parameters
split	divide a loop into several sub-loops -loop to split and split factors
fusion	merge several loops into a hyper-loop -adjacent loop to fuse
reorder	change execution orders of loops -loops to reorder and new order
unroll	unroll a loop by given depth -which loop to unroll and unroll depth
vectorize	apply vector operation to a loop -which loop to vectorize
inline	inline a function -which node to inline
compute_at	put producer in the body of consumer -which node and how deep to compute at
parallel	use multithreading-which loop to parallel
cache	use shared memory to store inputs/results -how much data to cache
bind	assign a loop to parallel blocks/threads -which loop to bind to block/thread

the basic unit in a block. A elastic MBConv cell is composed of sequential layers of *conv-1x1*, *dwconv-kxk* (depth-wise convolution with kernel size k), *SE*, and *conv-1x1*. Between *conv-1x1* and *dwconv-kxk*. In the elastic MBConv, the number of channels can be expanded by a ratio of E while the depth D can also be variable, which means only the first D units are kept in the sampled block. In addition to these configurable parameters, we allow a parameterisable kernel size through a kernel transformation matrix in each MBConv. Furthermore, we also allow the DNN model to take arbitrary input image sizes by assigning the model with a size ratio.

The elastic design of blocks offers sufficient flexibility for different deployment constraints. To reduce the evaluation cost, we train the super network that contains all the possible sub-networks through *weight sharing* and use it to estimate the accuracy of each sub-network.

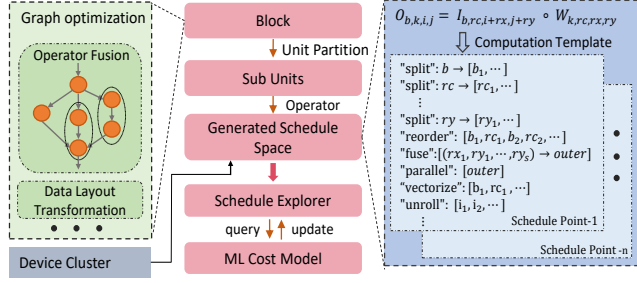


Figure 5. Block schedule optimization, including the graph-level optimization and lower-level operators schedule mapping.

3.4 Block-Level Pre-scheduling

In the prior NAS process, when a network architecture candidate is selected, it is mapped to the underlying hardware or the hardware model for performance evaluation. However, in CHaNAS, the generated network must try-out and choose the best scheduling policy for that particular hardware, and then evaluated for its true-optimal performance. Thereby, we must ensure its blocks are mapped to the hardware with the best scheduling policy for each network. To select the best scheduling point for a block on the target hardware, we build a block schedule optimizer $\mathcal{F}$ that takes the parametrizable block $block_i = (A_1, A_2, \dots, A_b)$ from the super-net and the hardware descriptor D_{hard} as inputs, and the output of the schedule optimizer is the best scheduling policy c_{block}^* and the associated network performance $P_{c_{block}^*}$ on the hardware:

$$c_{block}^*, P_{c_{block}^*} = \mathcal{F}(A_1, A_2, \dots, A_b, D_{hard}) \quad (2)$$

To guarantee efficient execution of these blocks for the target hardware. Figure 5 shows the process of block scheduling, which mainly comprises two parts: the graph-level optimization and the lower-level operators scheduling for the hardware platform. For graph-level optimization, we mainly adopt two methods: *layer fusion*, including intrinsic fusion, point-wise fusion, and kernel fusion, which reduces the memory and communication overhead for the intermediate data; *data layout transformation*, such as the flatten, concat and reorganization operation, which transform the feature data layouts into back-end friendly patterns.

For lower-level operators scheduling, as each block is composed of MBConv cells with similar structures but different tensor shapes, we optimize the schedule of each operator (e.g., conv2d, depth-wise conv2d) independently in the MB-Conv cell. The lower-level operator scheduling is an auto-tuning process that attempts to optimize the low-level implementations towards the peak hardware performance. Initially, We generate a scheduling space by enumerating different combinations of hardware-specific schedule primitives and the corresponding tunable parameters (listed in Table 1),

including the memory tiling, loop transformations, vectorization/tensorization, and parallelization strategies [13]. We then use a heuristic method to explore the scheduling space to obtain schedules that maximize performance for a given combination of operator, tensor shape, and the hardware configuration without human intervention.

Encoding the operator schedules. Each point in the scheduling space is encoded into a vector, and each element in the vector represents a specific choice of primitive or parameter. Figure 6 (c) and (e) are two scheduling examples for one conv2d operator instance in Figure 6 (b). Both (c) and (e) split the original seven loops of conv2d into thirteen sub-loops but with different split parameters, then reorder them and generate a larger out-most loop by fusion. Figure 6 (d) and (f) are the corresponding encoded representation of (c) and (e) respectively. The encoding follows a fixed order to ensure that the configuration points with the same number of parameters are put together. For operator split, we record the split factors, and create a cartesian product of sub-spaces. Suppose we splits a loop in the 2D convolution of size L into Z parts, different choices are generated via Z -factorization of integer S , and they can be represented as $[f_1, f_2, \dots, f_Z]$ where $f_1 \times f_2 \times \dots \times f_Z = L$. For reordering, we record the new order of loops. For fusion, the loops not recorded are meant to be fused with their neighboring outer loops. For unrolling, each loop corresponds to a value 0 (not to unroll) or 1 (unroll). Parallel and vectorize are not encoded as we always parallelize the outer-most loop and vectorize the inner-most loop in the baseline for demonstration.

Scheduling-level Search. To explore this large design space efficiently, we use a heuristic method to explore the scheduling space with a Gaussian-Process (GP) based cost model, which is pre-trained using runtime measurement data collected from the target hardware. Different points in the search space are evaluated by querying through the GP-based cost model. Before the scheduling strategy exploration, an initial set H is maintained, in which the scheduling points $p_1, p_2 \dots p_H$ and their performance result E_p are kept. In the scheduling-policy exploration process, We choose a starting point p in H with the probability $\exp^{-\gamma \frac{(E^* - E_p)}{E^*}}$ where γ is a hyperparameter, E^* is the performance of the best schedule point in H . The closer E_p is to E^* , the more likely p is chosen. We then take this point to generate a set of new points through mutation and evaluate these schedule points from the GP-based cost model. We also define some rules as a mutation mechanism to avoid the generation of inefficient schedule points. For example, we find that a split factor is efficient in most cases, while other split choices have inferior results. Hence, we limit the split factors to divisible split choices for each loop in the mutation process. Then the top-k newly-generated points will be chosen and evaluated on the hardware, and then added to H . All these performance measurement data are then used to update the

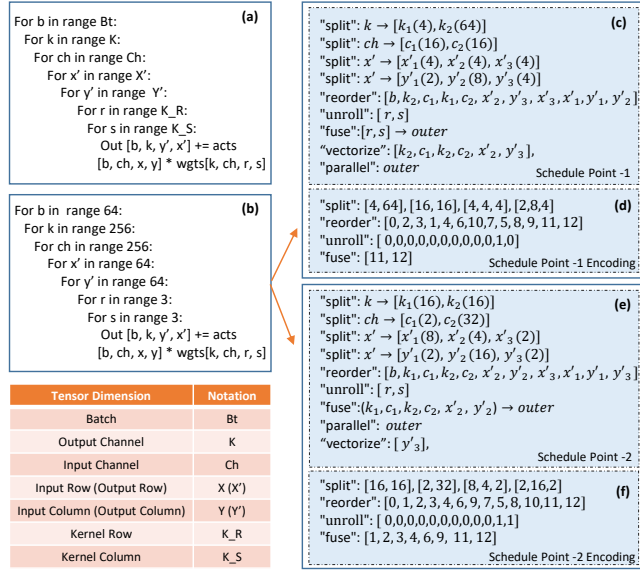


Figure 6. The example of Operator scheduling example in CHaNAS. (a) Conv2d description; (b) Conv2d code example; (c) Schedule description; (d) Schedule encoding of (c); (e) Schedule description; (f) Schedule encoding of (e).

GP-based cost model. Thanks to these blocks' structural similarity, we can reuse the GP-based cost model to speed up the scheduling-policy exploration. Consequently, we gradually find the optimal scheduling policy of an operator on the hardware platform after a certain number of iterations, which will be kept in a LUT for CHaNAS to pool during the coordinated search process.

3.5 Co-design Space Exploration

The size of the factorized hierarchical search space has been reduced exponentially when compared to the original joint space, since the candidate DNN architectures in the design space can be directly compared by their best performance, given the blocks that comprise the DNNs have already been evaluated on the target hardware by the pre-scheduling stage. However, the sub-net search space is still enormous; evaluating each candidate DNN still needs additional efforts even when we use the weight sharing to avoid the direct training. We first automatically divide the search space according to the performance requirements, which eliminate the search in the sub-space that does not satisfy the constraint. Then we use the evolutionary method to search for the target solution.

Automated NAS search space division. In our analysis, we find that *input resolution* and the *model expansion ratio* are the two most important factors affecting the network's performance, so we divide the network search space according to these two factors. In this work, we can select from an input resolution factor $In_size = \{160, 176, 192, 208, 224\}$ and

a model expansion ratio $Md_E = \{1.0, 1.1, 1.2, 1.3\}$ in our experiment. These parameters can be adjusted according to different situations. The space division leads to $In_size \times Md_E$ ($5 \times 4 = 20$ in our experiment) possible sub-spaces. Our first-step goal is to find the best sub-space that contains the target model.

To determine the target sub-space is non-trivial. One way is to perform network search on each of the sub search space and compare the final results. However, the computation overhead will be astronomical. Instead, we evaluate the quality of the search space by randomly sampling λ networks from the search space and comparing the distribution of the qualified networks. We collect the Cumulative Distribution Function (CDF) of each satisfying network's latency, and choose the sub search space with the lowest average latency. The intuition is that, with the same model family, the lower CDF of latency is more likely to achieve higher performance. For computing the network's latency, we construct a block performance LUT for the target device to enable fast performance estimation of DNN candidates. In the LUT, we track block-level latency metrics on real devices with different input dimensions, channel expansion ratio, etc. Thus, the latency of the network model can be obtained by referring to the block-level performance Lat_{block_i} in LUT Lat_{net} :

$$Lat_{net} = \sum_{i=1}^N Lat_{block_i} \quad (3)$$

Evolutionary search. After finding the best sub search space, we adopt an evolutionary search algorithm to find the target model efficiently. In evolutionary search process, each DNN architecture is represented by as a genome vector, denoted as $arch_i = [block_1, \dots, block_b] \in R_v$, where v is the length of the vector. In evolution iterations, we randomly choose 15K sub-networks with different architecture, and then measure their accuracy on 10K input samples from the validation set. To accelerate the evolution process, the $[arch_i, accuracy_i]$ pairs are used to train an MLP based accuracy predictor, which can quickly predict the model accuracy based on its architecture parameters. Through iteratively mutating high-quality DNN architectures, we can generate new DNN architectures of potentially higher quality. In each iteration, the searcher evaluates the fitness of each DNN architecture candidate based on the outputs from the accuracy and latency predictive models, then selects architectures with the highest fitness to breed the next generation using mutation and crossover operators. When a certain number of iterations is reached, or the constraints are satisfied, the searcher returns the DNN architecture from the evaluated set and the dedicated optimal scheduling policy for the target hardware.

Table 2. Performance Comparisons on Samsung Note10. CHaNAS-W/O indicates the solutions that are extracted from CHaNAS super-net without scheduling level re-optimization. “AWS Cost” is calculated based on the price of on-demand P3.16xlarge instances. N is the number of deployment scenarios we experimented in evaluation.

Model	ImageNet Top-1(%)	Mobile Latency	MACs	Search Cost (GPU hours)	NN Training Cost (GPU hours)	Total Cost (GPU hours)	AWS Cost
Mnasnet	74.0	34.4ms	317M	4000N	-	4000N	\$12250N
Fbnet-C	74.9	33.6ms	375M	216N	360N	576N	\$1764N
ProxylessNas-R	74.6	35.7ms	320M	200N	300N	500N	\$1530N
MobileNet-v3(large)	75.2	28.3ms	219M	-	180N	180N	\$550N
CHaNAS-W/O	76.0	27.5ms	224M	40N	1300	1300 + 40N	\$<124N
CHaNAS-W	76.2	16.4ms	240M	50N	1300	1300 + 50N	\$<150N

4 Evaluation

4.1 Experiments Setup

To demonstrate the efficacy of our proposed method, we evaluate CHaNAS by comparing it with four previous hardware-aware NAS works (Mnasnet [38], Fbnet [42], ProxylessNas [10], MobileNet-v3 [23]) on the ImageNet2012 classification dataset. As we mentioned before, we first build the elastic super-net that can be fit into different deployment constraint; then we optimize each possible block used in the super-net for the target hardware. At last, we reduce the co-design search space and use the evolutionary algorithm to perform the search. We use 5 blocks to form the super-net with each block has 4 MBConvs at most. Each MBConv has a kernel size within {3, 5, 7} and a channel expand ratio E within {3, 4, 6}. The input image size is ranged within {160, 176, 192, 208, 224} and the model expansion ratio ranges within {1.0, 1.1, 1.2, 1.3}. The standard SGD optimizer is used to train the supernet with Nesterov momentum 0.9 and weight decay $3e^{-5}$. The initial learning rate is 2.5, and the learning rate decays as the cosine function schedules. We also use the knowledge distillation technique to progressively fine-tune the sub-networks. The whole training process takes around 4.5 days on 16 NVIDIA P100 GPUs with 32GB memory each. For blocks compile optimization, we implement the optimization strategy in Python and use TVM [12] tools (version 0.7.dev) for code generation targeting various hardware platforms. It is noteworthy that the blocks optimization process is independent of the super-net training process. Thus, the whole blocks optimization process can be executed with the training process in parallel. In general, the time overhead of the pre-scheduling process can be hidden by training, and the joint-space search can be achieved with reasonable overhead, comparable to prior NAS methods that are agnostic to the scheduling optimization space.

We apply CHaNAS to three different hardware platforms: NVIDIA P100 GPU, Intel Xeon 8163 CPU@2.5GHz, and Samsung Note 10 phone (Snapdragon 855@2.8GHz). For comparison, the performance of previous works on GPUs are measured in Pytorch 1.3 + Cuda 10.1; the NAS solutions On

CPU are evaluated in Pytorch 1.3; On the mobile phone, the networks are implemented in TF_Lite. The performance is the averaged results of 1000 measurements with the workloads for more stable results.

4.2 Improvement from Scheduling-Level Optimization

To show the benefit of our block-based pre-schedule optimization, we provide the baseline implementation CHaNAS-W/O, which indicates the solutions that are extracted from CHaNAS super-net without scheduling level optimization. We measure the latency of their derived solutions in Pytorch on both GPU and CPU, and in TF_Lite on the mobile phone. CHaNAS-W presents the solutions that have gone through block-based scheduling-level optimization in the NAS process. We also use Note10 mobile phone as a hardware platform, as shown in Table 2, which reports the comparison between CHaNAS-W, CHaNAS-W/O, and previous hardware-aware NAS methods on Samsung Note 10 phone. As we can see, though CHaNAS-W has higher MACs than CHaNAS-W/O, it has an obvious reduction of inference latency (16.4ms vs 27.5ms), for the compiler-level co-design can dynamically adapt to different block structures and help search for the proper network schedules for the target hardware platform. In general, CHaNAS-W can achieve high performance as it strikes a balance between inter-thread and intra-thread workload decomposition on the ARM CPU by exploring numerous scheduling strategies.

Without pre-schedule optimization, CHaNAS-W/O achieves higher ImageNet top1 accuracy than MobileNet-v3 [23] with similar MACs. Specifically, CHaNAS-W/O achieves 1.2x, 1.4x, and 1.5x speedup than MobileNet-v3 [23] on GPU, CPU, and Mobile, respectively. This is attributed to the elastic super-net design; the sub-networks extracted from the CHaNAS super-net have not only high flexibility and medium-granularity suitable for hardware-oriented search.

4.3 Benefits Gained by Co-design

To prove the effectiveness of the coordinated search method, Figure 7 shows the Pareto-optimal points found by different hardware-aware NAS methods on three hardware platforms.

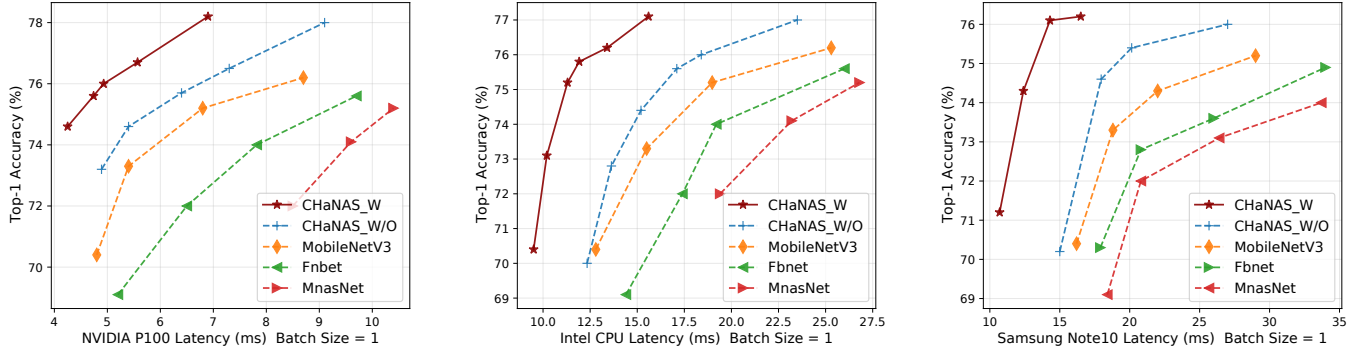


Figure 7. Neural network implementations provided by CHaNAS; Coordinated search achieve similar accuracy but significant speedup over the neural network solutions with a fixed scheduling policy on GPU, CPU, and Mobile, respectively.

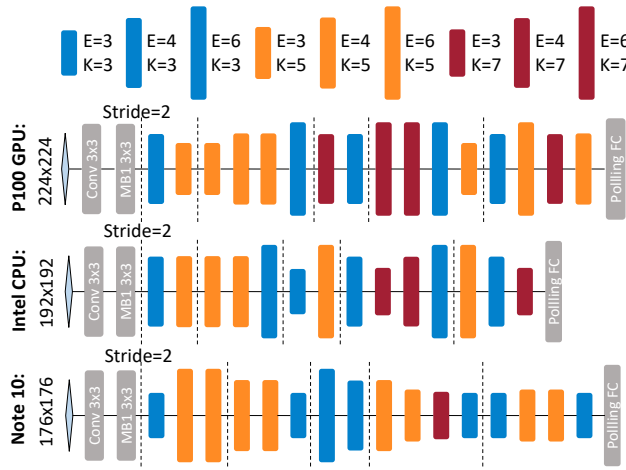


Figure 8. Visualization of the architectures found by CHaNAS for NVIDIA P100 GPU, Intel Xeon CPU, and Note 10 mobile.

Through a joint search of network architecture and scheduling policy, CHaNAS-W achieves the highest performance over all different cases of different performance requirements. CHaNAS-W achieves 78.4% ImageNet top1 accuracy On the P100 GPU, 2.3% more accurate than MobileNet-v3, the previous best result reported through hardware-aware NAS approach. Most importantly, CHaNAS-W runs 1.6x, 2.1x, 2.2x faster on the P100 GPU, 1.9x, 2.2x, 2.4x faster on the Intel Xeon 8163 CPU, and 1.7x, 2.7x, 2.8x faster on the Note 10 phone than MobileNet-v3, Fbnet, and Mnasnet respectively while delivering similar output accuracy. With the co-design approach, DNN solutions can be customized and optimized according to the target hardware to achieve better performance.

We also show the co-design cost of our CHaNAS compared with previous hardware-nas in Table 2 when developing the candidate network and schedules for N different application scenarios of different constraints; the search cost includes

pre-schedule optimization cost and co-search overhead before network deployment. “AWS Cost” is calculated based on the cloud-charging price of on-demand P3.16xlarge instances. Most previous hardware-nas need to re-design or re-train the candidate DNN model for a new hardware platform or even the change to the design constraint. For example, Mnasnet [38] needs 4000 GPU hours (near \$12250 AWS cost) for a new development scenario. While CHaNAS decouples the training and search process in NAS due to the block-based design, our super-net training can be performed only once and requires only a marginal search cost for fast deployment in a new application case. CHaNAS only need additional 50 GPU hours (\$150 AWS cost) for redeployment.

4.4 Analysis of Design-Points Found by CHaNAS

We visualize some of the discovered architectures for the three platforms in Figure 8 by using different colors and shapes to denote the kernel sizes and channel expansion ratios of MBConv. As shown in Figure 7, CHaNAS uses more 7x7 kernels in the neural network for GPU than other platforms, and we conjecture that CHaNAS can find more efficient GPU schedules for blocks with 7x7 kernels, due to the large parallelism of the computation array. CHaNAS can automatically find a solution that has higher arithmetic intensity to improve GPU utilization. On the Note 10 mobile phone, CHaNAS uses network models of smaller input size but with more layers than that on Intel CPU, which implies that it needs slimer but deeper architectures to maintain model accuracy due to the insufficiency of on-chip memory space in mobile phones. This is reasonable since deep-but-slim network architecture requires less on-chip memory space than the wider network architectures.

We also analyze the low-level operator schedule generation result found by the schedule search strategy. Take the Note 10 as a case, CHaNAS-W can enable register blocking through multi-level tiling and vectorize the inner-most loop to achieve vectorization, which are two critical factors to improve the schedule results on Note 10 phone. CHaNAS-W

can split spatial loops and reduce loops according to the split factors from exploration results, and uses split primitive and reorder primitive recursively to produce a series of small tiles to achieve the multi-level tiling. After splitting, the loops become smaller and can be potentially held by the cache to exploit data locality. To exploit parallelism, CHaNAS-W dynamically fuses several outer loops into one outer-most loop and parallelizes it. The performance improvement brought by CHaNAS-W shows our scheduling-level search process can dynamically optimize the code implementation in contrast to the previous fixed mapping strategy for various hardware and network architectures.

5 Related Work

Hardware-aware NAS. Recent hardware-aware NAS methods [21, 27, 38] directly incorporate the hardware feedback into the architecture search loop, to discover the neural networks that work best on specific hardware. For instance, Mnasnet [38] directly measures the inference latency by executing the model on mobile phones, so that it can employ the performance measurement to direct the model search process. OFA [9] can quickly search within a pre-trained supernet for a sub-graph network that achieves the required accuracy and speed on hardware. By these means, efficient network architectures with improved inference efficiency on particular hardware can be obtained automatically. However, prior hardware-aware NAS methods are oblivious to the compiler-level design choices, which means they do not consider the optimization of various scheduling policies when evaluating network candidates in the search process. Consequently, they may fail to find the de-facto optimal network model that must be eventually scheduled and mapped onto the hardware via a compiler. Hence, we believe automated network-model/scheduling-policy co-search is an approach towards better co-design solutions.

High-performance DNN model scheduling. Scheduling DNN model on the target hardware is another critical factor to improve the efficiency of the DNN system and has attracted a lot of attention from both academia and industry [19, 20]. The best practice currently is still developing schedule libraries for different hardware, and needs manually tuning for a new DNN model. Most existing deep learning frameworks [4, 11, 26, 35] rely on these libraries to achieve high performance. MKL-DNN [2] uses JIT techniques to optimize CNN on Intel Xeon CPUs. SWIRL [40] can generate high-quality fused, vectorized, and parallelized code for DNNs on CPUs. cuBlas [3] can accelerate linear algebra kernels to extreme high-performance, and cuDNN [17] accelerates deep learning applications by assembling a set of efficient algorithms on the GPUs. TF-Lite Micro [4] focuses on accelerating the DNN models on the embedded hardware. To adapt to more hardware back-ends, and improve the generality to different deep learning frameworks. Intel nGraph [18]

and Google XLA [1] have the role of a bridge between deep learning frameworks and hardware back-ends. Intel nGraph utilizes MKL-DNN to produce highly optimized implementations on CPUs and the PTX-emitting back-end of the LLVM to generate assembly code for GPUs. The XLA compiler acts as a back-end for TensorFlow. TVM [12] proposes an ahead-of-time compiler that supports multiple front-ends and hardware platforms. These compilers adopt high-level computing graphs and leverage fusion across layers based on predetermined rules. Besides, Auto-scheduling algorithms have gradually attracted a substantial amount of attention and provide appealing productiveness. Tensor Comprehension [39] adopts polyhedral IRs TC and employs a genetic search of affine transformation options (e.g., tile sizes, loop fusion, and scheduling strategies). PolyMage [34] introduces fusion methods with loop nests and determines the rules of fusion and the range of tiling sizes to ensure a small auto-tuning space. AutoTVM [12] utilizes high-level abstractions to represent the computing graph and includes a template-guided search algorithm for low-level tensor code generation. As a framework for generating tensor programs, FlexTensor [49] attempts to reduce human efforts in writing templates by using more general templates to cover more operators. To expand the optimization scope of tensor scheduling, Ansor [48] explores a larger search space to cover the useful tensor-level program optimization options. However, none of prior compiling or scheduling framework considers the joint search of both scheduling policy and network architecture.

Compared to traditional methods that either optimize the neural network using neural architecture search based on a fixed schedule strategy from a given deep learning library (e.g., Tensorflow[4], Pytorch [35]), or tunes the scheduling policy to maximize the inference speed for a given network [12, 13], CHaNAS can better utilize the resources by neural network architecture-schedule policy co-design.

6 Conclusion

We proposed an automated NAS framework, CHaNAS, to co-optimize the DNN architecture and the dedicated network scheduling policy for a specific machine learning task on the hardware. It directly involves compiler optimization in the DNN NAS loop, aiming to offer higher performance of DNN solutions for the target hardware platforms. With the proposed hierarchical co-design search space and the exploration method, both the network architecture design and the network scheduling can be effectively conducted on the basis of neural network blocks. When applied to the Imagenet2012 classification task on different hardware back-ends, CHaNAS can generate better co-design solutions over SOTA hardware-aware search methods MobileNet-v3. It is shown in the experiments that the co-design solutions generated by CHaNAS achieved 1.6x, 1.9x, and 1.7x performance boost respectively NVIDIA P100 GPU, Intel Xeon

CPU, and Samsung Note 10 Mobile, over the baselines of the same-level accuracy. We believe the larger neural network architecture and schedule co-design search space and effective exploration strategy could benefit the hardware-aware NAS methods further.

Acknowledgments

This work was supported by 1) the National Natural Science Foundation of China (No.61876173), 2) the Strategic Priority Research Program of Chinese Academy of Science (Grant No.XDC05030201), and 3) 2025 Key Technology Innovation Program of Ningbo City (No.2018B10035);

References

- [1] 2020. Google(R), XLA. <https://www.tensorflow.org/xla> (2020).
- [2] 2020. Inter(R) MKL-DNN. <https://github.com/intel/mkl-dnn> (2020).
- [3] 2020. NVIDIA(R), CUBLAS Library. <https://www.nvidia.com/> (2020).
- [4] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2017. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 17)*. 265–283. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [5] Mohamed S Abdelfattah, Łukasz Dudziak, Thomas Chau, Royson Lee, Hyeji Kim, and Nicholas D Lane. 2020. Best of both worlds: Automl codesign of a cnn and its hardware accelerator. In *57th ACM/IEEE Design Automation Conference (DAC)* (2020), 1–6. <https://doi.org/10.1109/DAC18072.2020.9218596>
- [6] Aayush Ankit, Abhronil Sengupta, and Kaushik Roy. 2018. Neuromorphic computing across the stack: Devices, circuits and architectures. In *2018 IEEE International Workshop on Signal Processing Systems (SiPS)*. IEEE, 1–6. <https://doi.org/10.1109/SiPS.2018.8598419>
- [7] Marco Bacis, Giuseppe Natale, Emanuele Del Sozzo, and Marco Domenico Santambrogio. 2017. A pipelined and scalable dataflow implementation of convolutional neural networks on FPGA. In *2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 90–97. <https://doi.org/10.1109/IPDPSW.2017.44>
- [8] Gabriel Bender, Pieter-Jan Kindermans, Barret Zoph, Vijay Vasudevan, and Quoc Le. 2018. Understanding and simplifying one-shot architecture search. In *International Conference on Machine Learning (ICML)* (2018), 550–559. <http://proceedings.mlr.press/v80/bender18a.html>
- [9] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. 2020. Once-for-all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations (ICLR)* (2020). <https://openreview.net/forum?id=HylxE1HKwS>
- [10] Han Cai, Ligeng Zhu, and Song Han. 2019. Proxylessnas: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR)* (2019). <https://openreview.net/forum?id=HylVB3AqYm>
- [11] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv:1512.01274* (2015). <http://arxiv.org/abs/1512.01274>
- [12] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated end-to-end optimizing compiler for deep learning. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*. 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [13] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. In *Conference on Neural Information Processing Systems (NeurIPS)* 31 (2018), 3389–3400. <https://proceedings.neurips.cc/paper/2018/hash/8b5700012be65c9da25f49408d959ca0-Abstract.html>
- [14] Weiwei Chen, Ying Wang, Shuang Yang, Chen Liu, and Lei Zhang. 2020. You Only Search Once: A Fast Automation Framework for Single-Stage DNN/Accelerator Co-design. In *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1283–1286. <https://doi.org/10.23919/DATE48585.2020.9116474>
- [15] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2017. Using dataflow to optimize energy efficiency of deep neural network accelerators. In *Proceedings of the 50nd Annual IEEE/ACM International Symposium on Microarchitecture (Micro)* 37, 3 (2017), 12–21. <https://doi.org/10.1109/MM.2017.54>
- [16] Yu-Hsin Chen, Tien-Ju Yang, Joel Emer, and Vivienne Sze. 2019. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. In *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* (2019), 292–308. <https://doi.org/10.1109/JETCAS.2019.2910232>
- [17] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. In *CoRR abs/1410.0759* (2014). <http://arxiv.org/abs/1410.0759>
- [18] Scott Cyphers, Arjun K Bansal, Anahita Bhiwandiwalla, Jayaram Bobba, Matthew Brookhart, Avijit Chakraborty, Will Constable, Christian Convey, Leona Cook, Omar Kanawi, et al. 2018. Intel ngraph: An intermediate representation, compiler, and executor for deep learning. *arXiv:1801.08058* (2018). <http://arxiv.org/abs/1801.08058>
- [19] Anup Das, Akash Kumar, and Bharadwaj Veeravalli. 2014. Energy-aware task mapping and scheduling for reliable embedded computing systems. In *ACM Transactions on Embedded Computing Systems (TECS)* 13, 2s (2014), 1–27. <https://doi.org/10.1145/2544375.2544392>
- [20] Anup Das, Akash Kumar, and Bharadwaj Veeravalli. 2015. Reliability and energy-aware mapping and scheduling of multimedia applications on multiprocessor systems. In *IEEE Transactions on Parallel and Distributed Systems* 27, 3 (2015), 869–884. <https://doi.org/10.1109/TPDS.2015.2412137>
- [21] Suyog Gupta and Berkin Akin. 2020. Accelerator-aware Neural Network Design using AutoML. *arXiv: 2003.02838* (2020). <https://arxiv.org/abs/2003.02838>
- [22] Cong Hao, Xiaofan Zhang, Yuhong Li, Sitao Huang, Jinjun Xiong, Kyle Rupnow, Wen-mei Hwu, and Deming Chen. 2019. FPGA/DNN Co-Design: An Efficient Design Methodology for IoT Intelligence on the Edge. In *56th ACM/IEEE Design Automation Conference (DAC)* (2019). <https://doi.org/10.1145/3316781.3317829>
- [23] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 1314–1324. <https://doi.org/10.1109/ICCV.2019.00140>
- [24] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861* (2017). <http://arxiv.org/abs/1704.04861>
- [25] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size. *arXiv:1602.07360* (2016). <http://arxiv.org/abs/1602.07360>
- [26] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*. 675–678. <https://doi.org/10.1145/2647868.2654889>

- [27] Weiwen Jiang, Xinyi Zhang, Edwin H-M Sha, Lei Yang, Qingfeng Zhuge, Yiyu Shi, and Jingtong Hu. 2019. Accuracy vs. Efficiency: Achieving Both through FPGA-Implementation Aware Neural Architecture Search. In *56th ACM/IEEE Design Automation Conference (DAC)* (2019), 1–6. <https://doi.org/10.1145/3316781.3317757>
- [28] Hyoukjun Kwon, Prasantha Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. 2019. Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (Micro)*. 754–768. <https://doi.org/10.1145/3352460.3358252>
- [29] Kiseok Kwon, Alon Amid, Amir Gholami, Bichen Wu, Krste Asanovic, and Kurt Keutzer. 2018. Co-design of deep neural nets and neural net accelerators for embedded vision applications. In *55th ACM/IEEE Design Automation Conference (DAC)* (2018), 1–6. <https://doi.org/10.1147/JRD.2019.2942284>
- [30] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2019. Darts: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)* (2019). <https://openreview.net/forum?id=5IeYHoC5FX>
- [31] Qing Lu, Weiwen Jiang, Xiaowei Xu, Yiyu Shi, and Jingtong Hu. 2019. On neural architecture search for resource-constrained hardware platforms. *arXiv:1911.00105* (2019). <http://arxiv.org/abs/1911.00105>
- [32] Yufei Ma, Yu Cao, Sarma Vrudhula, and Jae-sun Seo. 2017. Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 45–54. <http://dl.acm.org/citation.cfm?id=3021736>
- [33] Diana Marculescu, Dimitrios Stamoulis, and Ermao Cai. 2018. Hardware-Aware Machine Learning: Modeling and Optimization. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)* (2018). <https://doi.org/10.1145/3240765.3243479>
- [34] Ravi Teja Mullaipudi, Vinay Vasista, and Uday Bondhugula. 2015. Polyimage: Automatic optimization for image processing pipelines. In *ACM SIGARCH Computer Architecture News* 43, 1 (2015), 429–443. <https://doi.org/10.1145/2694344.2694364>
- [35] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *arXiv:1912.01703* (2019). <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [36] Ilija Radosavovic, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollár. 2020. Designing network design spaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10428–10436. <https://doi.org/10.1109/CVPR42600.2020.01044>
- [37] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Acm Sigplan Notices* 48, 6 (2013), 519–530. <https://doi.org/10.1145/2491956.2462176>
- [38] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le. 2019. Mnasnet: Platform-aware neural architecture search for mobile. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2820–2828. <https://doi.org/10.1109/CVPR.2019.00293>
- [39] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv:1802.04730* (2018). <http://arxiv.org/abs/1802.04730>
- [40] Anand Venkat, Tharindu Rusira, Raj Barik, Mary Hall, and Leonard Truong. 2019. SWIRL: High-performance many-core CPU code generation for deep neural networks. In *International Journal of High Performance Computing Applications* 33, 6 (2019), 1275–1289. <https://doi.org/10.1177/1094342019866247>
- [41] Ying Wang, Jie Xu, Yinhe Han, Huawei Li, and Xiaowei Li. 2016. Deep-Burning: Automatic generation of FPGA-based learning accelerators for the neural network family. In *53th ACM/IEEE Design Automation Conference (DAC)*. 1–6. <https://doi.org/10.1145/2897937.2898003>
- [42] Bichen Wu, Xiaoliang Dai, Peizhao Zhang, Yanghan Wang, Fei Sun, Yiming Wu, Yuandong Tian, Peter Vajda, Yangqing Jia, and Kurt Keutzer. 2019. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10734–10742. <https://doi.org/10.1109/CVPR.2019.01099>
- [43] Qingcheng Xiao, Yun Liang, Liqiang Lu, Shengen Yan, and Yu-Wing Tai. 2017. Exploring heterogeneous algorithms for accelerating deep convolutional neural networks on FPGAs. In *Proceedings of the 54th Annual Design Automation Conference 2017*. 1–6. <https://doi.org/10.1145/3061639.3062244>
- [44] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Bowen Shi, Qi Tian, and Hongkai Xiong. 2020. Latency-Aware Differentiable Neural Architecture Search. *arXiv: 2001.06392* (2020). <https://arxiv.org/abs/2001.06392>
- [45] Yifan Yang, Qijing Huang, Bichen Wu, Tianjun Zhang, Liang Ma, Giulio Gambardella, Michaela Blott, Luciano Lavagno, Kees Vissers, John Wawrzyniec, et al. 2019. Synetgy: Algorithm-hardware Co-design for ConvNet Accelerators on Embedded FPGAs. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)* (2019). <https://doi.org/10.1145/3289602.3293902>
- [46] Wang Ying, Xu Jie, Han Yinhe, Li Huawei, and Li Xiaowei. 2016. Deep-Burning: Automatic Generation of FPGA-based Learning Accelerators for the Neural Network Family. In *53th ACM/IEEE Design Automation Conference (DAC)* (2016). <https://doi.org/10.1145/2897937.2898003>
- [47] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 6848–6856. <https://doi.org/10.1109/CVPR.2018.00716>
- [48] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 19)*. 863–879. <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [49] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. 2020. FlexTensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 859–873. <https://doi.org/10.1145/3373376.3378508>
- [50] Barret Zoph and Quoc V Le. 2017. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)* (2017). <https://openreview.net/forum?id=r1Ue8Hcxg>